

Ask Data - Complete Project Guide

Generated on 2026-03-28 11:52:05

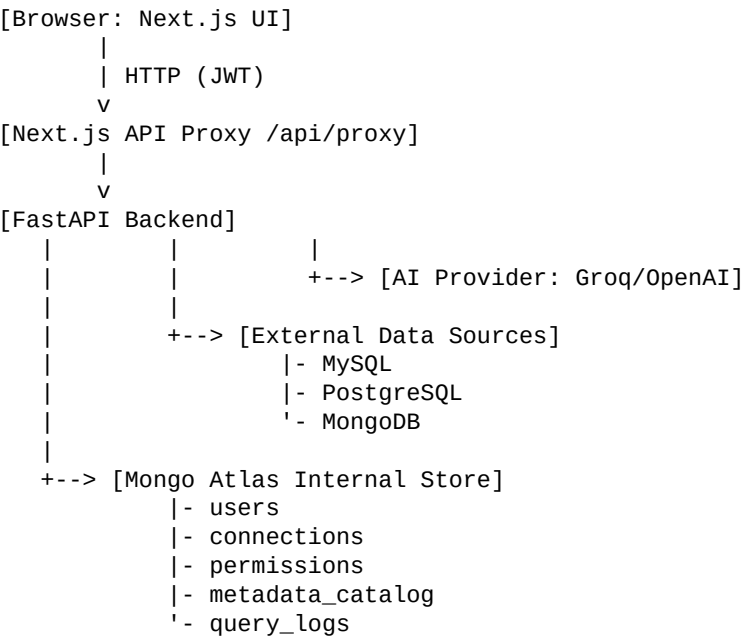
This document explains the full system architecture, connection flow, end-to-end workflows, and a file-by-file technical guide for the current codebase.

System Context

Layer	Technology	Purpose
Frontend	Next.js + React + Tailwind	Role-based UI (login/signup/admin/employee), chart rendering, API calls
Proxy	Next.js Route Handler	Forwards frontend requests to backend with fallback targets
Backend	FastAPI	Auth, RBAC, connection mgmt, metadata, query, export
Internal Store	MongoDB Atlas	Users, connections, permissions, metadata catalog, logs
External Sources	MySQL/PostgreSQL/MongoDB	Business data queried by users
AI Providers	Groq/OpenAI/Fallback	NL to query + overview generation

Architecture Diagrams

High-Level Architecture



Connection + Metadata Sequence

```
Admin UI -> Admin API (/admin/connections): create connection
Admin API -> encryption: encrypt credentials
Admin API -> system_store: save connection record
Admin API -> connection_service: test connection
Admin API -> metadata_service: refresh metadata
metadata_service -> external DB: inspect schema/collections
metadata_service -> metadata_catalog: save extracted fields
Admin API -> Admin UI: success + refreshed catalog
```

Employee Query Sequence

```
Employee UI -> Employee API (/employee/analyse): prompt + connection_ids
```

Employee API -> permissions store: validate allowed connections
Employee API -> ai_query_service: build execution plan
ai_query_service -> ai_service: generate SQL/logic (or fallback)
ai_query_service -> query executors: run read queries
ai_query_service -> Employee API: rows + overview + metadata
Employee API -> Employee UI: render table/chart + export options

Workflow Explanations

1) Authentication Workflow

- Admin registers via `/api/v1/auth/register`.
- Backend normalizes organisation/email, validates password strength, hashes password, stores user.
- User logs in via `/api/v1/auth/login`.
- Backend resolves organisation context and returns JWT with role claims.
- Frontend stores token and routes user to role-specific dashboard.

2) Employee Provisioning Workflow

- Admin opens User Management in Admin Console.
- Admin enters employee details and temporary password.
- Frontend blocks weak passwords with live strength meter.
- Backend endpoint `/api/v1/admin/employees` validates password and persists employee.
- Admin can then assign data permissions per connection.

3) Connection Onboarding Workflow

- Admin creates connection in form mode or URL mode.
- Credentials are encrypted before persistence.
- Connection test is executed to confirm reachability/authentication.
- Metadata refresh extracts schema/table/column information.
- Catalog is saved to metadata collection for downstream AI query context.

4) Permission Workflow

- Admin assigns connection-level permissions to each employee.
- Flags include `can_read`, `can_query`, `can_visualize`, `can_export`.
- Employee endpoints filter connections and operations by assigned permissions.
- Unauthorized actions are blocked at backend dependency/rule level.

5) Query and Analysis Workflow

- Employee submits natural-language prompt.
- AI query service picks provider (Groq/OpenAI) or fallback mode.
- Metadata context guides SQL generation or Mongo strategy.
- Only safe read semantics are allowed (SELECT-only for SQL paths).
- Results are returned as rows + optional overview + visualization-ready data.

6) Export Workflow

- User requests export as csv/excel/pdf from query or analysis result.
- Export service serializes current rows to selected format.

- Frontend receives blob and triggers file download.

Connection Modes and Data Paths

- Form mode: admin provides host/port/username/password/database_name fields.
- URL mode: admin provides complete connection_url; parser preserves database path and query options.
- Secrets are encrypted before persistence and decrypted only at runtime for test/query operations.
- Metadata refresh normalizes schema info into metadata_catalog used by AI and schema UI.
- Permission checks gate which connections an employee can read/query/visualize/export.

File-by-File Explanation

This section lists key files in the current repository and their responsibilities.

.env

Local environment configuration (secrets, URLs, keys).

.env.example

Template environment file with required keys and sample values.

README.md

Project overview, architecture summary, setup and runtime instructions.

backend/app/__init__.py

Code module used by this project. Review source for implementation specifics.

backend/app/api/__init__.py

Code module used by this project. Review source for implementation specifics.

backend/app/api/v1/__init__.py

Code module used by this project. Review source for implementation specifics.

backend/app/api/v1/admin.py

Admin features: employee management, data connections, permissions, metadata, analytics, exports.

backend/app/api/v1/auth.py

Register/login/logout endpoints, organisation-aware auth, password checks.

backend/app/api/v1/debug.py

Debug utilities and runtime diagnostics endpoints.

backend/app/api/v1/employee.py

Employee-facing routes: permitted connections, schema view, query/analyse/export actions.

backend/app/api/v1/query.py

Generic NL query and export endpoints with rate limiting.

backend/app/api/v1/router.py

Aggregates and mounts all versioned API routers.

backend/app/core/config.py

Environment settings and configuration defaults.

backend/app/core/dependencies.py

Auth dependencies and role guards (admin/employee).

backend/app/core/encryption.py

Encryption/decryption helpers for sensitive stored secrets.

backend/app/core/security.py

JWT creation/validation, password hashing and verification helpers.

backend/app/db/mongo.py

Mongo client setup, users collection access, and user lookup helpers.

backend/app/db/system_store.py

System collections wrappers (connections, metadata, logs, permissions, counters).

backend/app/main.py

FastAPI app bootstrap, middleware setup, health route, and API router wiring.

backend/app/models/__init__.py

Code module used by this project. Review source for implementation specifics.

backend/app/models/user.py

User model definitions used by backend domain.

backend/app/schemas/__init__.py

Code module used by this project. Review source for implementation specifics.

backend/app/schemas/auth.py

Pydantic request/response models for authentication.

backend/app/schemas/connection.py

Pydantic models for connection payloads and permission assignment.

backend/app/schemas/query.py

Pydantic models for query and export payloads.

backend/app/schemas/user.py

Pydantic user output models.

backend/app/services/__init__.py

Code module used by this project. Review source for implementation specifics.

backend/app/services/ai_query_service.py

Natural language to query pipeline execution.

backend/app/services/ai_service.py

AI provider selection, SQL/insight generation helpers, fallback logic.

backend/app/services/analysis_service.py

Multi-source analysis orchestration and result shaping.

backend/app/services/connection_service.py

Builds SQL/Mongo connection strings, validates and resolves target databases.

backend/app/services/export_service.py

CSV, Excel, and PDF export builders.

backend/app/services/metadata_service.py

Extracts schema metadata from SQL/Mongo sources into catalog.

backend/app/services/query_service.py

Query execution orchestration helpers.

backend/app/utils/password_validator.py

Backend password strength rules and detailed validation messaging.

frontend/app/admin/page.tsx

Admin console UI for users, connections, permissions, metadata, and analytics.

frontend/app/api/proxy/[...path]/route.ts

Next.js API proxy that forwards frontend calls to backend with fallback bases.

frontend/app/employee/page.tsx

Employee dashboard UI for querying and visual analysis.

frontend/app/layout.tsx

Global layout shell, metadata, and root styling wrappers.

frontend/app/page.tsx

Login page for admin/employee authentication.

frontend/app/signup-admin/page.tsx

Admin signup page with password strength checker and validation.

frontend/components/PasswordStrengthMeter.tsx

Reusable strength bar and checklist UI for password requirements.

frontend/components/ResultChart.tsx

Chart rendering component for query/analysis results.

frontend/lib/api.ts

HTTP client helper with auth header handling and error normalization.

frontend/lib/passwordValidator.ts

Frontend password strength scoring and requirement checks.

frontend/next-env.d.ts

Next.js TypeScript ambient type declarations.

frontend/tailwind.config.ts

Tailwind theme/content configuration.

How To Read This Project Quickly

1. Start with frontend/app/page.tsx and frontend/app/signup-admin/page.tsx for user entry points.
2. Inspect backend/app/api/v1/auth.py and backend/app/core/security.py for auth mechanics.
3. Move to frontend/app/admin/page.tsx and backend/app/api/v1/admin.py for core admin workflows.
4. Read backend/app/services/connection_service.py and metadata_service.py for data-source onboarding.
5. Read frontend/app/employee/page.tsx plus backend employee/query/analysis services for analytics flow.
6. Review frontend/lib/api.ts and proxy route for request routing and fallback behavior.
7. Finally, review password validators in frontend/lib/passwordValidator.ts and backend/app/utis/password_validator.py.

End of Document